

A thesis+antithesis=synthesis rumination on Contracts

Abstract

This paper provides a new analysis of the pros and cons of P2900 vs. an approach like P3640, explains how the latter would give us a much more manageable evolution path, and explains why we need to pull Contracts out of C++26 and put both of those approaches into a White Paper or a TS, and allow actual users to try out both.

Where does this come from? What's new?

What's new is an actual paper-analysis of two different evolutionary approaches, comparing two concrete proposals.

I looked at what [P3835](#) is hinting towards, contemplated the various feedback that has been given about the predictability, teachability, and usability of Contrats, and then looked at [P3640](#), and then came to the realization that we must ask ourselves the following question:

if, instead of maximum flexibility and coverage of all possible use cases, our focus is on simplicity of simple use cases, teachability, portability of source code, and predictability thereof, what things would we change?

Turns out, quite many. Some very significant ones.

But before going there, let's see what is the difference in approach and evolutionary paths we are talking about.

The beef

P2900	P3640
<code>void f(int x) pre(x >= 0); // a contract with flexible semantics</code>	<code>void f(int x) pre(x >= 0); // a contract with fixed semantics</code>
<code>void f(int x) pre<somelabel>(x >= 0); // a contract with possibly fixed semantics</code>	<code>void f(int x) pre<somelabel>(x >= 0); // a contract with possibly flexible semantics</code>

The first row shows the starting point, the second row shows a possible future extension. In

- P2900, the starting point is that the semantics are flexible, and subject to change via compiler flags and violation handler behavior.
- P3640, the starting point is that the semantics are fixed.

In

- P2900, the future extension is that you can specify more-exact and more-fixed semantics for a contract.
- P3640, the future extension is that you can specify a contract to have more-flexible semantics.

In

- P2900, the shortest syntax is the one that brings in all the bells and whistles.
- P3640, the shortest syntax is the simple one, and it and its meaning are easy to understand.

So what would ***I*** do differently with a different focus?

There's a simple and straightforward example of what I would do differently with a different focus. Consider the paper I wrote about allowing double-evaluation of contract predicates, [P3264](#).

That paper is all about flexibility. It's all about being able to turn contract checks on and off on different sides of an app/library boundary, and its focus is plainly on the train of thought of not particularly caring whether the contracts are on or off until you need to find a bug. In order to do that, it avoids ABI breaks introduced by the switching of contracts on and off like a plague. It's not worried about any guarantees, or about any sort of "safety".

If the focus is on simplicity, predictability, and guarantees of what source code means, that picture changes ***completely***. With that focus, you don't write

```
void f(int x) pre(x >= 0);
```

and then make that have different meanings based on compiler flags. You either write

```
void f(int x) pre(x >= 0);
```

or you write

```
void f(int x);
```

and you do not switch between them all the time. You stick to one, and enjoy its guarantees (with the P3640 approach).

And from ***that*** it follows that there's no particular reason those two alternatives should be ODR-equivalent or ABI-compatible. They just aren't. Not even when they are two definitions of the same inline function.

What problems does the P3640 approach avoid?

Looking at the various papers, and P3835 and P3640 in particular, we can make some very interesting observations. To simplify the analysis further, consider that we take the P3640 approach so that the only evaluation semantic available is `quick_enforce`, and thus there isn't even any violation handler to consider.

We find that

- The optimization concern discussed in [P3829](#) doesn't come up. Regardless of whether we agree on refinement problems being compiler bugs, the question never arises. There is no refinement. There is no mixed modes. The concern becomes moot.
- The mixed-mode concerns, which are not refinement concerns, discussed in P3835 don't come up. There is no mixed modes, all TUs behave the same way, all definitions of inline functions behave the same way, both for `contract_asserts` and preconditions.
- The audiences who want guaranteed contract checks get exactly what they want, with the simplest syntax imaginable.

- The audiences concerned about teachability, explicability, portability of source code and its meanings, and understability of code using contracts have a **much** simpler facility to teach, explain, understand, and use.

There are, of course, various concerns that need to be talked about:

- There is no 'observe' semantic. However, we can provide that with flexible semantics later, with a different syntax. The practical uses of an 'observe' semantic tend to be such that they really do need a flexible semantic anyway, either to look at whether a contract can be narrowed, or whether it can be widened. You start by making the semantic flexible, then have users run your code with different semantics, and find out the results, and then maybe later choose a different fixed semantics if nothing broke.
- If the chosen fixed semantic is `quick_enforce`, there is no violation handler. You can't make contract violations throw exceptions, or otherwise prevent termination. Those abilities can also be added later, with a different syntax.
- If the chosen fixed semantic is `enforce`, there is a violation handler, but it's unclear whether that would be the best choice for a fixed semantic if, in addition to simplicity, explicability, and ease of use our focus is on safety.

Some will then say that not having a violation handler from the get-go is unacceptable. They don't seem to be correct. Fedora, as a particular trailblazer, and other Linux distros following, has been packaging all of its C++ code for years built so that hardening is on, so C++ standard library functions that have hardening support forcibly and abruptly terminate your program if you violate their hardened preconditions. It's effectively shipping what P3640 proposes. That's existing shipping practice.

That's been out in the wild for years. Every Fedora user has used that, happily. Very few if any have even noticed. There's been no noticeable performance regressions, nor rampant terminations of applications. The sky didn't fall.

In addition, having that **as the default** doesn't prevent programmers from building their `stdlib`-using applications with library hardening turned off. Vendors of other libraries can provide the same ability. And later, when the facility evolves to allow flexible semantics, library vendors can adopt that if they want to.

The problem with P2900, and C++26, i.e. the status quo

There are claims that P2900 is just an MVP, and it allows us to gather feedback, and is therefore good to ship in C++26.

But it can't actually do that. We **will** get feedback along the lines that the short syntax should do what P3640 does. We are already getting that feedback. In some cases, we won't get such feedback because the people trying this stuff out will not believe that we can make such a change once we have shipped the short-syntax-means-flexible-semantics in C++26, because that would be a breaking change.

And they are right. Even some of us who would think that's the right approach to take, and the right thing to do would be to make that breaking change are going to have to oppose such a breaking change.

P2900 is a trial balloon. We do not have deployment experience worth mentioning on **function contract assertions**. We have some for assertion-statements, but none by actual third-party users.

Some are saying we can't get meaningful feedback unless we ship in an IS, because, paraphrasing, "nobody will do large-scale in-production deployments with an experimental feature".

Well, first of all, that's evidently and demonstrably incorrect; we did get such deployments for Coroutines and Modules, and for different approaches for Modules. We got smaller-scale deployments for Concepts, multiple third-party libraries adopted Concepts, that's why we have Ranges in C++20.

That's also a self-fulfilling prophecy. We can't get the feedback we seek on two different approaches if we tie the talons of our experimental-approach eagle to the floor. It will never soar, we make that impossible. We can have academic groups and industry groups experiment and give us feedback, but due to shipping the facility in an IS, we can't break it. That's The End of useful deployment experiments, before they had a chance to begin.

And we haven't made those experiments and investigations feasible thus far. In [P2899](#), the deployment experience section states thus:

"Implementations of [P2900R14] are available in GCC and Clang."

That is nonsense. That is not correct. The implementations are available in **forks** of those implementations. The vast majority of this committee doesn't know how to build them, and doesn't know how to find out how to use them. Less-expert users would have even less of a chance.

The shipping releases of those compilers don't have an implementation of P2900, nor do even the git snapshots of those compilers. We haven't yet achieved what we should achieve to make deployments feasible in order to ascertain that our feature is good. It's a major feature, and not having that deployment experience before standardising it is unacceptable, full stop.

P3640 isn't "just an if-statement"

Before we go to a suggested solution, I must discuss one more thing. There's very frequently comments about contracts that have a guaranteed checked semantic, comments that in a downright derisive tone state "you don't need that, you already have that functionality, just write an if".

That, I must unfortunately state, is the biggest colossal misconception I have ever heard in the world of standardization. It's **nothing** like an if-statement, it's not like an if-statement **at all**.

It's a very common practice in the industry, especially in mixed-expertise development teams, that more-experienced software engineers design APIs, and less-experienced ones implement them.

A guaranteed precondition (or a postcondition) allows such an API designer to ensure that that check is there. The less-experienced developer can't forget it, and can't remove it. Using an if-statement doesn't have that advantage; the less-experienced developer might not know to add one. They might mistakenly remove one when refactoring a function definition. When the guaranteed pre/post are used on a function declaration, they can't accidentally remove those, such changes will get caught. Changes deep in definitions might not get caught so easily.

The solution

We need to, we have to, we must

1. remove Contracts from the C++26 working draft
2. put both P2900 and P3640 into a White Paper or a Technical Specification, so that the simple syntax is available for both, with an implementation-defined mechanism to switch what that syntax means, i.e. an implementation-defined mechanism to switch between P2900 and P3640
3. we need to encourage our implementation vendors to upstream such an implementation into their subsequent releases
4. and then we need to encourage our members who are University teachers to run deployment experiments of both with their students, and our industry members to run deployment experiments at whatever scale they can

We can't afford to ship a contracts facility in a C++ IS without running that experiment. Our users can't afford that. Our ecosystem can't afford that.

Additional note: we do **not** need a fully-fledged "labels" facility to run that experiment. It's a White Paper, it doesn't need all the bells and whistles, and the syntax it uses doesn't need to be final. We can do something along the lines of

P2900	P3640
<code>void f(int x) pre(x >= 0); // a contract with flexible semantics</code>	<code>void f(int x) pre(x >= 0); // a contract with fixed semantics</code>
<code>void f(int x) pre fixed(x >= 0); // a contract with the same fixed semantics as the fixed ones in P3640</code>	<code>void f(int x) pre flex(x >= 0); // a contract with flexible semantics, i.e. what P2900 does with its plain syntax</code>

and that will allow us to experiment on this particular, but fundamentally important, aspect equally well as we could with a fully-fledged "labels" facility.